

NGCF with Attention-Augmented Message Passing

1 Model Formulation: Neural Graph Collaborative Filtering (NGCF)

We consider a bipartite graph $G_r = (U, I, E)$, where $U = \{u_1, \dots, u_N\}$, $I = \{i_1, \dots, i_M\}$, and edges $E \subseteq U \times I$ represent user-item interactions. Each node has an embedding vector $\mathbf{e}_u, \mathbf{e}_i \in \mathbb{R}^d$, initialized randomly or using side attributes.

Message Construction and Propagation

At layer l , the embedding of user u is:

$$\mathbf{e}_u^{(l)} = \sigma \left(\mathbf{m}_{u \leftarrow u}^{(l)} + \sum_{i \in \mathcal{N}_u} \mathbf{m}_{u \leftarrow i}^{(l)} \right), \quad (1)$$

where σ is the LeakyReLU activation. The message from item i to user u is:

$$\mathbf{m}_{u \leftarrow i}^{(l)} = \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \left(\mathbf{W}_1^{(l)} \mathbf{e}_i^{(l-1)} + \mathbf{W}_2^{(l)} (\mathbf{e}_i^{(l-1)} \odot \mathbf{e}_u^{(l-1)}) \right), \quad (2)$$

and the self-message:

$$\mathbf{m}_{u \leftarrow u}^{(l)} = \mathbf{W}_1^{(l)} \mathbf{e}_u^{(l-1)}. \quad (3)$$

Matrix Propagation

NGCF propagation in matrix form:

$$\mathbf{E}^{(l)} = \sigma \left((\mathbf{L} + \mathbf{I}) \mathbf{E}^{(l-1)} \mathbf{W}_1^{(l)} + (\mathbf{L} \mathbf{E}^{(l-1)} \odot \mathbf{E}^{(l-1)}) \mathbf{W}_2^{(l)} \right), \quad (4)$$

where $\mathbf{L} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ is the normalized adjacency matrix.

2 Attention-Augmented Message Construction (Our Work)

We enhance NGCF with an attention mechanism that learns the importance of each neighbor during aggregation.

User-Side Aggregation with Attention

$$\alpha_{u,i}^{(l)} = \frac{\exp \left(\mathbf{v}_a^\top \tanh \left(\mathbf{W}_a [\mathbf{e}_u^{(l-1)} \parallel \mathbf{e}_i^{(l-1)}] \right) \right)}{\sum_{j \in \mathcal{N}_u} \exp \left(\mathbf{v}_a^\top \tanh \left(\mathbf{W}_a [\mathbf{e}_u^{(l-1)} \parallel \mathbf{e}_j^{(l-1)}] \right) \right)}, \quad (5)$$

$$\mathbf{e}_u^{(l)} = \sigma \left(\sum_{i \in \mathcal{N}_u} \alpha_{u,i}^{(l)} \mathbf{e}_i^{(l-1)} \right) \quad (6)$$

Item-Side Aggregation with Attention

$$\alpha_{i,u}^{(l)} = \frac{\exp\left(\mathbf{v}_a^\top \tanh\left(\mathbf{W}_a[\mathbf{e}_i^{(l-1)} \parallel \mathbf{e}_u^{(l-1)}]\right)\right)}{\sum_{j \in \mathcal{N}_i} \exp\left(\mathbf{v}_a^\top \tanh\left(\mathbf{W}_a[\mathbf{e}_i^{(l-1)} \parallel \mathbf{e}_j^{(l-1)}]\right)\right)}, \quad (7)$$

$$\mathbf{e}_i^{(l)} = \sigma\left(\sum_{u \in \mathcal{N}_i} \alpha_{i,u}^{(l)} \mathbf{e}_u^{(l-1)}\right) \quad (8)$$

Intuition

- Learns soft relevance scores between center and neighbors.
- Avoids noisy or uninformative neighbors.
- Captures personalized aggregation patterns.

3 Implementation Highlights and Limitations

Efficient PyTorch Aggregation

In code, our aggregation function uses matrix operations:

- Neighbor embeddings are retrieved using `'index_select'`. *Pairwise attention scores are recomputed for* `[e_i || e_j]` using `'matmul'`.
- A softmax is applied across each row (attention weights per neighbor set).
- Weighted sum is performed using `'torch.bmm()'` (batched matrix multiplication).

Limitations of Fixed-Size Neighbor Sampling

To simplify tensor computations, we fix the number of neighbors to a constant k via padding or trimming. While efficient, this has drawbacks:

- Wastes compute on dummy neighbors for sparse nodes.
- Prevents dynamic attention masking—no support for variable neighborhood sizes.
- Could bias learning toward denser users/items.

Mask-aware implementations (as in PyTorch Geometric or Deep Graph Library) can address this but are more complex to vectorize.

4 Background: Attention Mechanisms

In general, attention assigns relevance to inputs. For a query q , key k , and value v :

$$\text{Attention}(q, k, v) = \text{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)v \quad (9)$$

In GAT (Graph Attention Network), attention is learned via:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{a}^\top [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k \in \mathcal{N}(i)} \exp\left(\text{LeakyReLU}\left(\vec{a}^\top [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_k]\right)\right)} \quad (10)$$

5 Algorithm: Attention-based Aggregation

Algorithm 1 Attention-based Aggregation for Node i

Require: Embeddings $\{e_j\}$, center node e_i , neighbors $\mathcal{N}(i)$, weights $\mathbf{W}_a, \mathbf{v}_a$

```

1: for each  $j \in \mathcal{N}(i)$  do
2:   Concatenate:  $\mathbf{c}_{ij} = [e_i || e_j]$ 
3:   Hidden rep:  $\mathbf{s}_{ij} = \tanh(\mathbf{W}_a \mathbf{c}_{ij})$ 
4:   Score:  $\alpha_{ij} = \mathbf{v}_a^\top \mathbf{s}_{ij}$ 
5: end for
6: Normalize:  $\alpha_{ij} = \text{softmax}_j(\alpha_{ij})$ 
7: Aggregate:  $e_i^{\text{agg}} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} e_j$ 
8: return  $e_i^{\text{agg}}$ 

```

Comparison: Attention-based Aggregation vs Laplacian-based Propagation

NGCF combines two key paradigms in graph-based learning: (1) **Spectral Graph Convolution** via Laplacian propagation and (2) **Edge-wise Message Construction** between users and items. Our work introduces a third component—**Node-wise Attention Aggregation**—to selectively weight neighbor contributions.

Laplacian-based Propagation (Original NGCF). The propagation of embeddings is driven by the normalized adjacency matrix $\mathbf{L} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$, as follows:

$$\mathbf{E}^{(l)} = \sigma \left((\mathbf{L} + \mathbf{I}) \mathbf{E}^{(l-1)} \mathbf{W}_1^{(l)} + (\mathbf{L} \mathbf{E}^{(l-1)} \odot \mathbf{E}^{(l-1)}) \mathbf{W}_2^{(l)} \right) \quad (11)$$

This formulation captures higher-order connectivity across the entire graph through matrix multiplication, propagating embeddings across multiple hops via stacked layers. However, it uses a fixed normalized weighting scheme, treating all neighbors equally after degree adjustment.

Attention-based Aggregation (Our Extension). In contrast, our approach integrates an attention mechanism at each layer, assigning learnable weights to each neighbor based on the embedding context:

$$\alpha_{u,i}^{(l)} = \frac{\exp \left(\mathbf{v}_a^\top \tanh \left(\mathbf{W}_a [\mathbf{e}_u^{(l-1)} || \mathbf{e}_i^{(l-1)}] \right) \right)}{\sum_{j \in \mathcal{N}_u} \exp \left(\mathbf{v}_a^\top \tanh \left(\mathbf{W}_a [\mathbf{e}_u^{(l-1)} || \mathbf{e}_j^{(l-1)}] \right) \right)} \quad (12)$$

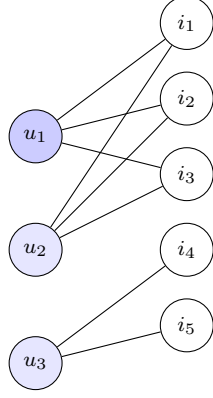
$$\mathbf{e}_u^{(l)} = \sigma \left(\sum_{i \in \mathcal{N}_u} \alpha_{u,i}^{(l)} \mathbf{e}_i^{(l-1)} \right) \quad (13)$$

This formulation allows node-specific prioritization of neighbors, enhancing local adaptivity. It is applied separately for user-side and item-side aggregations and replaces fixed aggregation with learned relevance.

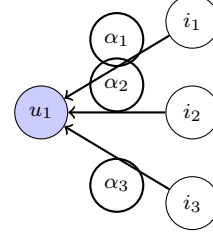
Unified Interpretation. Our attention-augmented NGCF combines:

- **Laplacian-based propagation:** Encodes high-order graph structure through spectral smoothing.
- **Attention-based aggregation:** Captures semantic neighbor importance within each user/item’s local 1-hop neighborhood.

While the Laplacian operator provides global smoothing and propagation, the attention mechanism allows fine-grained control over local influence, especially when attributes are included or neighborhood quality varies.



(a) NGCF: High-order connectivity via Laplacian propagation.



(b) Our Attention-enhanced NGCF: Neighbor contributions weighted by $\alpha_{u,i}$

Figure 1: Comparison of message aggregation strategies: (a) uses spectral GCN with implicit higher-order propagation, (b) explicitly models attention weights from neighbors.

6 Conclusion: Integrating Attention into NGCF

Our NGCF variant introduces attention-enhanced neighbor aggregation. It learns dynamic relevance across users/items, improves expressiveness, and supports attribute-based embeddings. While current implementation uses fixed-size neighbors for simplicity, future work may explore dynamic masked attention over variable-length neighborhoods.